

## Capitulo-02-Seccion-01-v1.0

---

# Anatomía del Contexto

---

*Módulo 3 — Context Engineering Profesional*

---

## Un problema concreto

---

Imagine un asistente corporativo que responde correctamente por la mañana y comienza a fallar por la tarde.

Después de revisar el modelo se descubre que el problema no está en el LLM.

La causa es otra:

- documentos obsoletos recuperados por RAG;
- memoria persistente desactualizada;
- historial excesivamente largo;
- herramientas devolviendo datos inconsistentes.

Al finalizar este capítulo, el lector dispondrá del vocabulario y los criterios para diagnosticar exactamente este tipo de problema. Más importante aún, sabrá cómo evitarlo desde el diseño.

---

## ¿Por qué hablar de anatomía?

---

Así como un ingeniero civil debe conocer las partes que componen una estructura antes de diseñar un edificio, un ingeniero de IA necesita comprender cada componente del contexto antes de construir un asistente, un agente o una aplicación basada en LLM.

La mayoría de los problemas observados en producción no se originan en el modelo, sino en un contexto incompleto, contradictorio o mal organizado.

---

## Una visión de alto nivel

---

Podemos representar el recorrido de una consulta mediante el siguiente esquema:



Cada bloque aporta información distinta y cumple una responsabilidad específica. Las próximas secciones analizarán cada uno en profundidad.

---

## Objetivos del capítulo

---

Al finalizar este capítulo el lector será capaz de:

- identificar cada componente del contexto;
  - comprender cómo interactúan entre sí;
  - detectar errores de diseño;
  - evaluar arquitecturas de contexto desde una perspectiva profesional.
-

## Resumen

---

El contexto no es un elemento único sino un ecosistema formado por múltiples componentes que evolucionan continuamente. El trabajo del ingeniero de IA consiste en diseñar, coordinar y mantener ese ecosistema para que el modelo disponga siempre de la información adecuada.

---

## Capitulo-02-Seccion-02-v1.0

---

# El ciclo de vida del contexto

---

*Módulo 3 — Context Engineering Profesional*

---

# Introducción

---

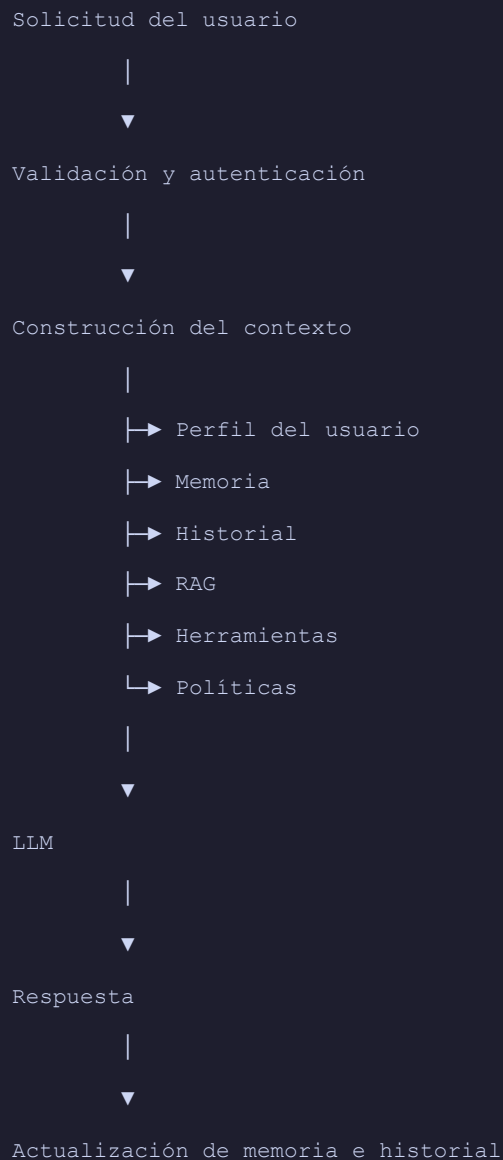
Una de las ideas más importantes del Context Engineering es que **el contexto no es estático**. Se crea, se enriquece, se transforma y finalmente se descarta o resume. Comprender este ciclo de vida es esencial para diseñar aplicaciones con LLM que sean eficientes y predecibles.

---

## El contexto como un flujo

---

En una aplicación empresarial, el contexto atraviesa varias etapas antes de llegar al modelo.



Cada fase puede modificar el contexto antes de la inferencia.

---

## Etapa 1 – Recepción

---

El sistema recibe la solicitud y obtiene información básica:

- identidad del usuario;
- canal de acceso;
- fecha y hora;
- permisos disponibles.

En esta etapa todavía no interviene el modelo.

---

## Etapa 2 – Enriquecimiento

---

La aplicación incorpora información adicional:

- memoria persistente;
- historial reciente;
- documentos recuperados mediante RAG;
- resultados de consultas a APIs;
- políticas internas.

El objetivo es entregar al modelo únicamente la información relevante.

---

## Etapa 3 – Inferencia

---

Con el contexto construido, el LLM genera una respuesta.

En este punto el modelo no consulta nuevas fuentes por sí mismo: razona sobre el contexto recibido.

Por ello, cualquier omisión o contradicción afecta directamente la calidad del resultado.

---

## Etapa 4 – Persistencia

---

Una vez obtenida la respuesta, el sistema decide qué conservar.

No todo debe almacenarse.

Ejemplos de información persistente:

- nuevas preferencias del usuario;
- decisiones relevantes;
- tareas pendientes.

En cambio, mensajes triviales o datos temporales pueden descartarse.

---

## Riesgos comunes

---

Un mal diseño del ciclo de vida suele producir:

- historiales interminables;
- memorias desactualizadas;
- documentos duplicados;
- aumento del costo por tokens;
- respuestas inconsistentes.

Estos problemas no suelen resolverse cambiando el modelo, sino rediseñando el flujo del contexto.

---

## Buenas prácticas

---

- Construir el contexto justo antes de la inferencia.
  - Mantener separadas memoria e historial.
  - Recuperar conocimiento bajo demanda.
  - Eliminar información irrelevante.
  - Persistir solo aquello que aporte valor futuro.
-

## Resumen

---

El contexto tiene un ciclo de vida completo: nace con la solicitud del usuario, se enriquece con distintas fuentes, alimenta al modelo y finalmente deja información útil para futuras interacciones. Diseñar correctamente este ciclo es una de las responsabilidades centrales del ingeniero de IA.

La siguiente sección analizará en detalle el papel de las instrucciones del sistema como la capa más estable y determinante del contexto.

---

## Capitulo-02-Seccion-03-v1.0

---

# Las instrucciones del sistema: el ADN del contexto

---

*Módulo 3 — Context Engineering Profesional*

---

# Introducción

---

Si el contexto fuese una organización, las instrucciones del sistema serían su estatuto. Definen las reglas permanentes de funcionamiento del modelo y establecen los límites dentro de los cuales deberá responder.

En la mayoría de las aplicaciones empresariales, esta es la capa más estable y una de las más importantes de toda la arquitectura.

---

## ¿Qué son las instrucciones del sistema?

---

Las instrucciones del sistema son mensajes preparados por la aplicación antes de incorporar el mensaje del usuario.

Su finalidad es definir:

- el rol del modelo;
- el idioma de respuesta;
- el nivel técnico esperado;
- restricciones de seguridad;
- formato de salida;
- objetivos del asistente.

Estas instrucciones no deberían contener información específica de una conversación, sino reglas generales de comportamiento.

---

# Responsabilidades

---

Una buena instrucción del sistema responde preguntas como:

- ¿Quién sos?
- ¿Cómo debés responder?
- ¿Qué nunca debés hacer?
- ¿Qué formato debés utilizar?
- ¿Qué prioridad tienen otras instrucciones?

Separar estas decisiones del resto del contexto simplifica el mantenimiento de la solución.

---

## Ejemplo

---

En lugar de escribir en cada consulta:

*"Actúa como un arquitecto de software, responde en español, utilizá Markdown y justificá las decisiones."*

Una aplicación profesional incorpora estas reglas una sola vez dentro de la capa de sistema.

El usuario únicamente expresa su necesidad.

---

## Qué NO debería incluir

---

Evite colocar en las instrucciones del sistema:

- datos temporales;
- resultados de APIs;
- documentos RAG;
- historial conversacional;
- preferencias específicas de un usuario.

Esos elementos pertenecen a otras capas del contexto.

---

## Diseño recomendado

---

Una estructura habitual incluye:

1. Identidad del asistente.
2. Objetivo principal.
3. Restricciones.
4. Políticas de seguridad.
5. Formato esperado.
6. Criterios de calidad.

Este orden facilita la lectura y el mantenimiento.

Los criterios de calidad establecen el estándar mínimo que debe cumplir cada respuesta: por ejemplo, exactitud técnica, completitud de la información solicitada y tono apropiado para el público destinatario. Definirlos explícitamente permite evaluar la salida del modelo de forma consistente.

---

## Error frecuente

---

Un error común consiste en convertir las instrucciones del sistema en un documento enorme que intenta resolver todos los escenarios posibles.

Esto produce:

- mayor consumo de tokens;
- contradicciones;
- dificultad para evolucionar la aplicación.

Las reglas permanentes deben ser pocas, claras y estables.

---

## Nota del arquitecto

---

Cada vez que una regla cambia con frecuencia, pregúntese si realmente pertenece a la capa de sistema o debería convertirse en contexto dinámico.

Las capas estables y dinámicas no deberían mezclarse.

---

## Resumen

---

Las instrucciones del sistema constituyen el fundamento sobre el cual se construye el resto del contexto. Definen la identidad del asistente y garantizan un comportamiento consistente entre conversaciones.

En la próxima sección analizaremos el contexto de ejecución y cómo incorporar información dinámica sin comprometer la estabilidad de la arquitectura.

---

## Capitulo-02-Seccion-04-v1.0

---

# El contexto de ejecución

---

*Módulo 3 — Context Engineering Profesional*

---

# Introducción

---

Las instrucciones del sistema proporcionan estabilidad, pero por sí solas no alcanzan para responder correctamente en un entorno empresarial. Cada interacción ocurre dentro de una situación concreta que cambia continuamente.

**El contexto de ejecución** es el conjunto de datos temporales que describen el estado actual de la aplicación, del usuario y del entorno en un instante determinado. A diferencia de las instrucciones del sistema, estos datos pueden variar entre dos solicitudes consecutivas y carecen de valor una vez finalizada la operación.

---

# Características

---

El contexto de ejecución posee cuatro propiedades fundamentales:

1. **Temporal:** cambia constantemente.
  2. **Específico:** pertenece a una única interacción o sesión.
  3. **Derivado:** suele obtenerse de otros sistemas.
  4. **Descartable:** gran parte deja de tener valor al finalizar la operación.
-

## ¿Qué contiene?

---

Ejemplos habituales de datos de ejecución:

- usuario autenticado;
  - rol y permisos;
  - organización;
  - idioma de la sesión;
  - fecha y hora;
  - ubicación;
  - estado de un proceso;
  - identificadores de recursos.
-

## Ejemplo práctico

---

Supongamos una aplicación de gestión documental.

El usuario solicita:

*"Mostrame los expedientes pendientes."*

La aplicación incorpora automáticamente:

- usuario: María Gómez;
- organismo: Dirección Jurídica;
- permisos: lectura y aprobación;
- fecha actual;
- expediente activo;
- filtros de seguridad.

El LLM ya no responde a una pregunta genérica, sino a una consulta contextualizada.

---

# Separación de responsabilidades

Una arquitectura saludable mantiene separados:

| Componente            | Responsabilidad         |
|-----------------------|-------------------------|
| Sistema               | Reglas permanentes      |
| Contexto de ejecución | Estado actual           |
| Memoria               | Información persistente |
| Historial             | Conversación vigente    |
| RAG                   | Conocimiento recuperado |

Esta separación reduce el acoplamiento y facilita la evolución del sistema.

## Errores frecuentes

---

Entre los problemas más habituales se encuentran:

- incluir datos temporales dentro del prompt del sistema;
- reutilizar contexto de una sesión anterior;
- enviar permisos incorrectos;
- no validar la identidad del usuario antes de construir el contexto.

Cada uno de estos errores puede producir respuestas incorrectas o incluso incidentes de seguridad.

---

## Buenas prácticas

---

- Construir el contexto inmediatamente antes de la inferencia.
  - Validar siempre autenticación y autorización.
  - Incorporar únicamente información necesaria para la solicitud actual.
  - Evitar duplicar datos presentes en otras capas.
-

## Resumen

---

El contexto de ejecución conecta el mundo real con el modelo de lenguaje. Es la capa que adapta las reglas generales del sistema a la situación concreta de cada interacción.

En la próxima sección analizaremos cómo el historial conversacional aporta continuidad y cuáles son las estrategias para administrarlo sin superar los límites de la ventana de contexto.

---

## Capitulo-02-Seccion-05-v1.0

---

# El historial conversacional

---

*Módulo 3 — Context Engineering Profesional*

---

# Introducción

---

Una de las capacidades que más valor aportan a los asistentes modernos es la posibilidad de mantener una conversación coherente a lo largo del tiempo. Esa continuidad no surge por arte de magia: depende de cómo se construye y administra el historial conversacional.

El historial es una parte del contexto, pero no debe confundirse con la memoria. Mientras la memoria conserva información de largo plazo, el historial representa la secuencia de intercambios de la conversación actual.

---

## ¿Qué es el historial conversacional?

---

Es el registro ordenado de los mensajes intercambiados entre el usuario, el modelo y, en muchos casos, las herramientas utilizadas durante una sesión.

Su función principal es permitir que el modelo interprete referencias implícitas, mantenga el hilo de la conversación y evite solicitar información que ya fue proporcionada.

---

## ¿Por qué es importante?

---

Sin historial, el modelo trataría cada consulta como un evento aislado.

Por ejemplo:

Usuario:

*"Mostrame el ticket 1532."*

Luego:

*"¿Quién lo creó?"*

La segunda pregunta solo puede responderse correctamente si el modelo conserva el contexto de la primera.

---

## Qué suele contener

---

Un historial bien diseñado puede incluir:

- mensajes del usuario;
- respuestas del asistente;
- llamadas a herramientas;
- resultados relevantes de esas herramientas;
- eventos importantes de la conversación.

No todo debe conservarse. La información redundante o sin valor futuro puede resumirse o eliminarse.

---

## Crecimiento del historial

---

Uno de los principales desafíos es que el historial crece con cada interacción.

Si se envían todos los mensajes al modelo:

- aumenta el consumo de tokens;
- se incrementa el costo;
- disminuye el espacio disponible para nueva información;
- aparecen contradicciones y ruido.

Por ello, las aplicaciones empresariales implementan mecanismos para controlar su tamaño.

---

## Estrategias habituales

---

Las más utilizadas son:

1. **Ventana deslizante:** conservar únicamente los últimos mensajes.
2. **Resumen conversacional:** reemplazar intercambios antiguos por un resumen.
3. **Historial híbrido:** combinar mensajes recientes con resúmenes y eventos clave.
4. **Recuperación inteligente:** reincorporar conversaciones anteriores solo cuando sean relevantes.

Cada estrategia presenta ventajas y compromisos que se analizarán con mayor profundidad en capítulos posteriores.

---

## Error frecuente

---

Guardar absolutamente toda la conversación rara vez constituye una buena práctica.

El objetivo no es conservar la mayor cantidad posible de texto, sino preservar la información necesaria para que el modelo continúe razonando correctamente.

---

## Buenas prácticas

---

- Definir una política de retención.
  - Resumir conversaciones extensas antes de que saturen la ventana de contexto.
  - Separar claramente historial y memoria.
-

## Resumen

---

El historial conversacional aporta continuidad a la interacción, pero debe administrarse cuidadosamente para evitar costos innecesarios y pérdida de calidad. Diseñar una estrategia adecuada de gestión del historial constituye una competencia esencial dentro del Context Engineering.

En la próxima sección estudiaremos la memoria persistente y veremos cómo complementa al historial para construir asistentes capaces de recordar información relevante entre conversaciones.

---

## Capitulo-02-Seccion-06-v1.0

---

# La memoria persistente

---

*Módulo 3 — Context Engineering Profesional*

---

# Introducción

---

Mientras que el historial conversacional permite mantener el hilo de una sesión, la **memoria persistente** permite que un sistema recuerde información relevante entre conversaciones diferentes.

Esta capacidad es uno de los elementos que distingue a un asistente moderno de un simple chatbot.

---

## ¿Qué es la memoria persistente?

---

La memoria persistente es el conjunto de datos que una aplicación conserva más allá de una conversación individual para mejorar futuras interacciones.

A diferencia del historial, no representa una secuencia cronológica de mensajes, sino conocimiento útil sobre el usuario, el dominio o la aplicación.

---

## ¿Qué puede recordar un asistente?

---

Dependiendo del caso de uso, la memoria puede almacenar:

- preferencias del usuario;
- idioma habitual;
- zona horaria;
- proyectos activos;
- decisiones previas;
- configuraciones frecuentes;
- estilo de comunicación.

En un entorno empresarial también puede registrar:

- equipos asignados;
  - áreas de trabajo;
  - autorizaciones;
  - procesos iniciados;
  - contexto de negocio.
-

## ¿Qué NO debería almacenarse?

---

No toda la información merece convertirse en memoria.

Evite persistir:

- datos temporales;
- resultados de consultas puntuales;
- información desactualizada;
- mensajes triviales;
- contenido redundante.

Una memoria excesiva termina degradando la calidad del contexto.

---

# Tipos de memoria

---

## Memoria del usuario

Describe características relativamente estables del usuario: preferencias, idioma, zona horaria, estilo de comunicación.

## Memoria de la aplicación

Conserva estado y decisiones relevantes para el funcionamiento del sistema: procesos iniciados, configuraciones activas, autorizaciones vigentes.

## Memoria del dominio

Almacena información específica del negocio que puede reutilizarse en múltiples conversaciones: definiciones propias de la organización, jerarquías, convenciones internas.

Cada tipo posee un ciclo de vida diferente y requiere políticas propias de actualización e invalidación.

---

## Ciclo de vida

---

Una implementación profesional responde cuatro preguntas:

1. ¿Qué información se almacena?
2. ¿Cuándo se actualiza?
3. ¿Cuándo deja de ser válida?
4. ¿Quién puede utilizarla?

Responder estas preguntas evita memorias inconsistentes y problemas de seguridad.

---

## Caso práctico

---

En un asistente corporativo, un usuario indica que prefiere recibir las respuestas en español técnico y en formato Markdown.

Esa preferencia puede almacenarse como memoria del usuario.

En futuras conversaciones ya no será necesario solicitarla nuevamente.

Sin embargo, el estado de un ticket consultado hace una semana probablemente no deba conservarse como memoria: es un dato temporal sin valor para interacciones futuras.

---

## Buenas prácticas

---

- Definir criterios claros de persistencia antes de implementar.
  - Versionar las estructuras de memoria cuando evolucionen.
  - Permitir actualizar o eliminar información de forma explícita.
  - Evitar almacenar datos sensibles sin una justificación clara.
-

## Error frecuente

---

Un error habitual consiste en utilizar la memoria como un depósito ilimitado de información.

La memoria no debe crecer indefinidamente. Debe permanecer útil, consistente y relevante para las tareas futuras.

---

## Resumen

---

La memoria persistente aporta continuidad entre conversaciones y permite personalizar la experiencia del usuario. Diseñarla correctamente requiere definir políticas de creación, actualización y eliminación, diferenciándola claramente del historial conversacional.

En la siguiente sección estudiaremos cómo interactúan memoria, historial y recuperación de conocimiento para construir un contexto completo y equilibrado.

---

## Capitulo-02-Seccion-07-v1.0

---

# Memoria, historial y RAG: cuándo utilizar cada uno

---

*Módulo 3 — Context Engineering Profesional*

---

# Introducción

---

Uno de los errores más comunes al diseñar aplicaciones con modelos de lenguaje consiste en utilizar indistintamente memoria, historial y RAG como si fueran mecanismos equivalentes.

Aunque los tres forman parte del contexto, **resuelven problemas completamente diferentes**. Elegir el componente incorrecto suele producir aplicaciones más costosas, menos precisas y difíciles de mantener.

---

# Tres componentes, tres responsabilidades

| Componente | Propósito principal                            | Duración           | Pregunta que responde                             |
|------------|------------------------------------------------|--------------------|---------------------------------------------------|
| Historial  | Mantener el hilo de la conversación actual     | Minutos u horas    | ¿Qué ocurrió en esta conversación?                |
| Memoria    | Recordar información útil entre conversaciones | Días, meses o años | ¿Qué conviene recordar del usuario o del sistema? |
| RAG        | Recuperar conocimiento externo actualizado     | Bajo demanda       | ¿Qué conocimiento necesito consultar ahora?       |

## Ejemplo práctico

---

Un usuario pregunta:

*"Necesito continuar el informe que empezamos ayer."*

La aplicación actúa de la siguiente manera:

- El **historial** aporta las últimas interacciones de la conversación.
- La **memoria** recuerda que el usuario suele trabajar en español y prefiere informes en Markdown.
- El **RAG** recupera la documentación del proyecto almacenada en la base de conocimiento.

El modelo recibe un contexto mucho más rico que si dependiera únicamente del mensaje del usuario.

---

## Un criterio sencillo

---

Antes de incorporar información al contexto, pregúntese:

1. ¿Solo sirve para esta conversación? → Historial.
2. ¿Será útil en conversaciones futuras? → Memoria.
3. ¿Proviene de documentos o datos externos? → RAG.

Este criterio evita gran parte de los errores de diseño.

---

## El caso borde: información que pertenece a más de una categoría

---

Un escenario frecuente en producción surge cuando una pieza de información podría pertenecer a más de una categoría simultáneamente.

Por ejemplo: durante una conversación, el equipo de arquitectura decide adoptar un nuevo estándar de nomenclatura para los servicios. Esa decisión es parte del historial de la sesión pero también debería quedar registrada en la base de conocimiento de la organización para ser consultada en el futuro.

El criterio general para estos casos es:

- si la información tiene valor operacional inmediato solo para esta conversación, va al historial;
- si tiene valor estructural y reutilizable para el sistema o para el negocio, va a RAG;
- si tiene valor personal y persistente sobre el usuario, va a memoria.

Cuando la información cumple más de un criterio, registrarla en ambos lugares es válido, siempre que exista una política clara sobre cuál es la fuente de verdad y cómo se mantienen sincronizadas.

---

## Errores frecuentes

---

### Guardar documentos completos en memoria

La memoria debe contener conocimiento persistente y estructurado, no grandes volúmenes de documentación. Los documentos extensos pertenecen a RAG.

### Usar el historial como base documental

El historial no reemplaza una base de conocimiento. A medida que crece, se vuelve costoso e ineficiente.

### Recuperar siempre los mismos documentos

El RAG debe recuperar únicamente la información relevante para la consulta actual, no un conjunto fijo de documentos por defecto.

---

## Patrón recomendado

---

Una arquitectura empresarial suele seguir este flujo:

```
Usuario
|
├─ Historial reciente
├─ Memoria persistente
├─ Recuperación RAG
└─ Herramientas
    |
    ▼
    Contexto final
    |
    ▼
    LLM
```

Cada componente aporta información diferente sin superponer responsabilidades.

---

## Buenas prácticas

---

- Mantener separados los tres mecanismos con políticas de actualización independientes.
  - Evitar duplicar información entre memoria y RAG sin una justificación explícita.
  - Priorizar calidad antes que cantidad en cada capa.
-

## Ejercicio de diagnóstico

---

Un equipo implementa un asistente de atención al cliente con la siguiente arquitectura:

- El historial de la sesión actual se almacena junto con conversaciones de los últimos 30 días.
- La documentación del producto (manual de 400 páginas) se guarda completa en la memoria persistente del usuario.
- Las instrucciones del sistema incluyen: nombre del usuario autenticado, permisos de acceso y la hora de inicio de la sesión.
- Cuando el usuario consulta el estado de un pedido, el sistema recupera los mismos 10 documentos de la base RAG sin importar la consulta.

Identifique los errores de diseño aplicando los criterios aprendidos en este capítulo.

### Errores esperados:

1. El historial de 30 días mezcla historial conversacional con memoria; los intercambios antiguos deberían resumirse o persistirse como memoria estructurada.
  2. La documentación completa del producto pertenece a RAG, no a memoria persistente; almacenarla en memoria satura el contexto y lo encarece.
  3. El nombre del usuario, los permisos y la hora de sesión son datos de contexto de ejecución, no reglas del sistema; incluirlos en el prompt de sistema los vuelve estáticos e incorrectos en la próxima sesión.
  4. El RAG debería recuperar únicamente los documentos relevantes para la consulta actual, no siempre el mismo conjunto fijo.
-

## Resumen

---

Historial, memoria y RAG no compiten entre sí: se complementan. Una arquitectura de Context Engineering madura utiliza cada mecanismo para aquello que mejor sabe hacer, logrando asistentes más precisos, eficientes y fáciles de evolucionar.

En la próxima sección analizaremos el papel de las herramientas y los resultados de ejecución como parte de la anatomía del contexto.

---

## Capitulo-02-Seccion-08-v1.0

---

# Herramientas como parte del contexto

---

*Módulo 3 — Context Engineering Profesional*

---

# Introducción

---

Los primeros modelos de lenguaje solo podían responder utilizando la información con la que habían sido entrenados y el contexto recibido. Hoy, los LLM modernos pueden interactuar con herramientas externas para consultar información, ejecutar acciones y enriquecer el contexto antes de producir una respuesta.

En Context Engineering, las herramientas no son un complemento: forman parte de la arquitectura del contexto.

---

## ¿Qué entendemos por herramienta?

---

Una herramienta es cualquier mecanismo externo que permite al modelo obtener información o ejecutar una acción que no puede realizar por sí mismo.

Ejemplos habituales:

- APIs REST;
- bases de datos;
- sistemas ERP y CRM;
- motores de búsqueda;
- servicios de correo electrónico;
- calendarios;
- repositorios de documentos;
- sistemas de tickets.

El modelo no accede directamente a estos recursos. Es la aplicación la que ejecuta la herramienta y devuelve el resultado al contexto.

---

## Flujo de trabajo

---

Una interacción típica sigue este esquema:



El dato importante es que el modelo razona sobre el **resultado** de la herramienta, no sobre la herramienta en sí.

---

## Ejemplo

---

Un usuario pregunta:

*"¿Cuántos incidentes críticos siguen abiertos?"*

El modelo no conoce esa información.

La aplicación:

1. consulta el sistema de tickets;
  2. obtiene el listado actualizado;
  3. incorpora el resultado al contexto;
  4. solicita al LLM que genere una respuesta comprensible.
-

# Herramientas versus conocimiento

---

Una herramienta responde preguntas sobre el estado actual del mundo.

Un documento RAG responde preguntas sobre conocimiento.

Por ejemplo:

- Manual de uso → RAG.
- Estado de un ticket → Herramienta.
- Política institucional → RAG.
- Cantidad de usuarios activos → Herramienta.

Comprender esta diferencia evita arquitecturas innecesariamente complejas.

---

## Manejo de errores

---

Un aspecto crítico en arquitecturas de agentes es el tratamiento de los errores devueltos por las herramientas. Si una API no responde, devuelve un código de error o retorna datos vacíos, el resultado que llega al contexto no es información útil sino señal de falla.

Un modelo que recibe ese resultado sin contexto adicional puede interpretarlo de formas impredecibles: inventar una respuesta, reportar un resultado incorrecto o bloquear el flujo por completo.

La aplicación debe normalizar siempre el resultado antes de incorporarlo al contexto. Esto incluye:

- detectar y registrar el código de error;
- decidir si reintentar o degradar la respuesta;
- informar al modelo de manera explícita qué ocurrió.

Por ejemplo, si el sistema de tickets no responde, el contexto debería incluir:

```
[Resultado de herramienta: get_open_incidents]
Estado: error
Código: 503
Mensaje: "El servicio no está disponible en este momento."
```

Con esa información, el modelo puede responder con precisión: "No pude obtener los incidentes abiertos porque el sistema de tickets está temporalmente fuera de servicio." Esto preserva la confianza del usuario y evita respuestas inventadas.

---

## Errores frecuentes

---

- Enviar resultados excesivamente largos al modelo sin filtrar.
- No validar errores devueltos por una API antes de incorporarlos al contexto.
- Confiar ciegamente en información incompleta o potencialmente incorrecta.
- Ejecutar herramientas cuando el contexto ya contiene la respuesta.

Cada llamada tiene un costo y un impacto sobre la latencia.

---

## Buenas prácticas

---

- Normalizar los resultados antes de enviarlos al LLM.
  - Manejar errores y tiempos de espera de forma explícita y descriptiva.
  - Evitar consultas redundantes cuando la información ya está disponible en el contexto.
-

## Nota del arquitecto

---

Las herramientas no reemplazan al modelo y el modelo no reemplaza a las herramientas.

Una arquitectura profesional combina ambos componentes para aprovechar las fortalezas de cada uno.

---

## Resumen

---

Las herramientas permiten que un modelo interactúe con sistemas externos y trabaje con información actualizada. Sus resultados pasan a formar parte del contexto y amplían significativamente las capacidades de una solución basada en IA. El manejo correcto de errores es tan importante como el de los resultados exitosos.

En la próxima sección estudiaremos cómo la seguridad, las políticas y las restricciones también forman parte de la anatomía del contexto y condicionan el comportamiento del modelo.

---

## Capitulo-02-Seccion-09-v1.0

---

# Políticas, restricciones y seguridad del contexto

---

*Módulo 3 — Context Engineering Profesional*

---

# Introducción

---

En una aplicación empresarial no basta con proporcionar información al modelo. También es necesario definir **qué información puede utilizar, qué acciones están permitidas y qué límites nunca debe sobrepasar.**

Las políticas y restricciones forman parte del contexto porque condicionan directamente el comportamiento del modelo durante la inferencia.

---

## La seguridad comienza antes del LLM

---

Un error frecuente consiste en asumir que el modelo será quien haga cumplir las reglas de negocio.

En realidad, la responsabilidad principal recae sobre la aplicación.

Antes de construir el contexto deben verificarse:

- identidad del usuario;
- autenticación;
- autorización;
- pertenencia a una organización;
- permisos sobre los recursos solicitados.

El LLM debe recibir únicamente la información que el usuario está autorizado a conocer. Garantizar esto también protege frente a técnicas como el *prompt injection*, donde un atacante intenta manipular el comportamiento del modelo a través del contenido del contexto. Este tipo de ataques se analizará en profundidad en el Capítulo 14 (Seguridad, Gobernanza y Compliance).

---

# Tipos de restricciones

---

Una arquitectura profesional suele combinar distintos niveles de control:

## Restricciones funcionales

Determinan qué operaciones puede realizar el asistente.

Ejemplos:

- solo consulta;
- consulta y creación;
- aprobación de solicitudes.

## Restricciones de datos

Limitan qué información puede incorporarse al contexto.

Por ejemplo:

- expedientes de un organismo;
- tickets de un área;
- documentos clasificados.

## Restricciones de formato

Definen cómo debe responder el modelo.

Ejemplos:

- Markdown;
  - JSON válido;
  - tablas;
  - lenguaje técnico.
-

## El principio del mínimo privilegio

---

Una regla ampliamente utilizada en ciberseguridad consiste en otorgar únicamente los permisos necesarios para realizar la tarea.

Aplicado al Context Engineering significa:

- recuperar solo los documentos necesarios;
- consultar únicamente las APIs requeridas;
- enviar al modelo la mínima cantidad de datos sensibles.

Reducir el contexto también reduce la superficie de exposición.

---

## Información sensible

---

Es recomendable evitar incorporar al contexto:

- contraseñas;
- claves API;
- secretos;
- tokens de autenticación;
- datos personales innecesarios;
- información financiera sin justificación.

Cuando estos datos sean imprescindibles, deben protegerse mediante controles adicionales definidos por la aplicación.

---

## Caso práctico

---

Supongamos un asistente de Recursos Humanos.

Dos usuarios realizan exactamente la misma consulta:

*"Mostrame el legajo de Juan Pérez."*

Aunque el prompt sea idéntico, el contexto será diferente.

- Un analista de RR. HH. puede acceder al expediente completo.
- Un gerente puede visualizar solo determinados campos.
- Un empleado común no debería recibir información confidencial.

La diferencia no está en el modelo, sino en el contexto construido por la aplicación.

---

## Buenas prácticas

---

- Validar permisos antes de recuperar información.
  - Evitar mezclar información de distintos niveles de confidencialidad en el mismo contexto.
  - Revisar periódicamente las políticas de acceso a medida que el sistema evoluciona.
-

## Resumen

---

Las políticas y restricciones constituyen una capa esencial de la anatomía del contexto. Permiten adaptar el comportamiento del modelo a las reglas del negocio y proteger la información sensible antes de que llegue al LLM.

En la próxima sección integraremos todos los componentes estudiados para construir un modelo conceptual completo de la anatomía del contexto.

---

## Capitulo-02-Seccion-10-v1.0

---

# Integración de la anatomía del contexto

---

*Módulo 3 — Context Engineering Profesional*

---

# Introducción

---

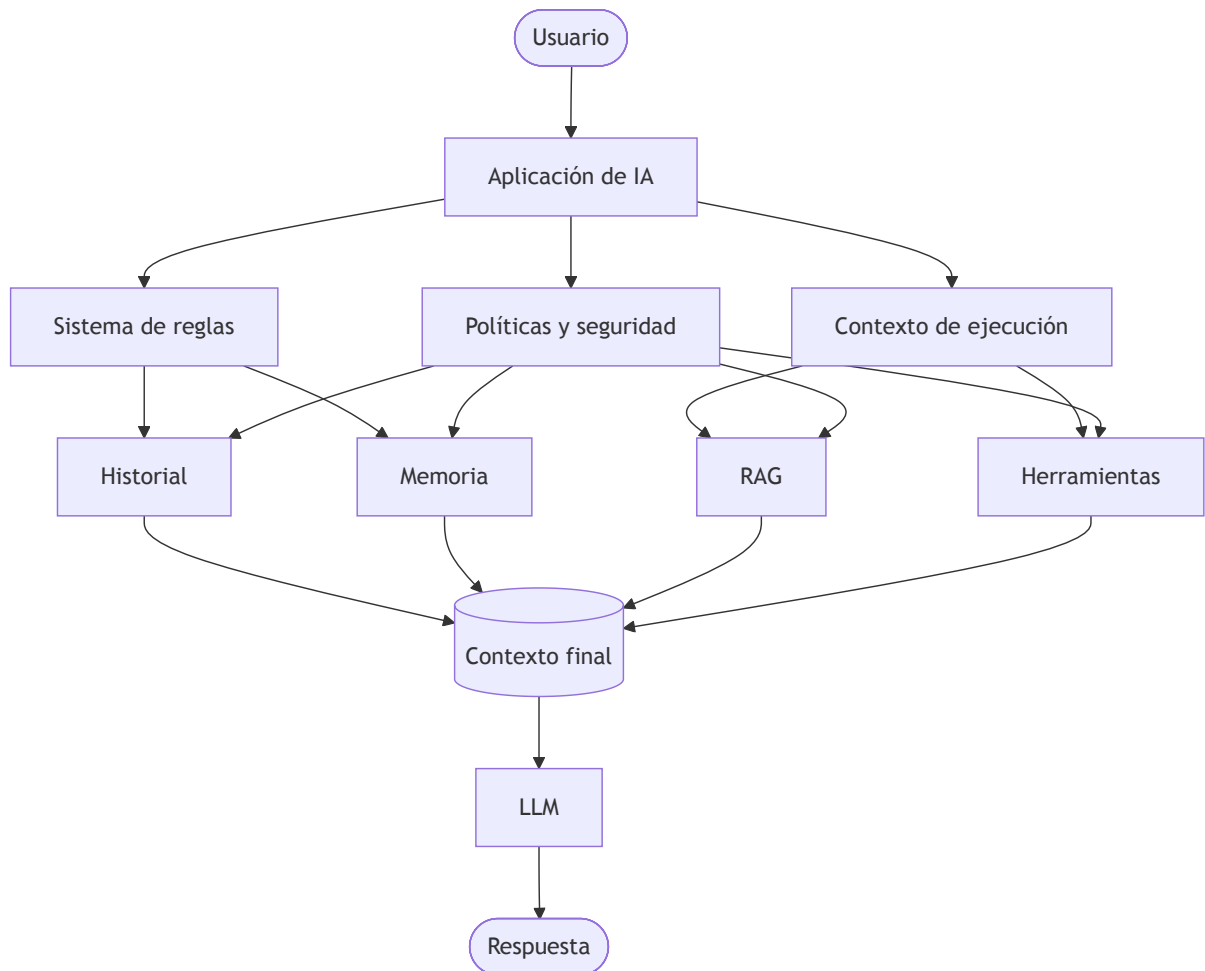
A lo largo de este capítulo analizamos cada uno de los componentes que forman el contexto de un modelo de lenguaje. En esta sección final los integraremos en una visión unificada para comprender cómo colaboran durante una interacción real.

El objetivo de un ingeniero de IA no consiste únicamente en conocer cada componente por separado, sino en diseñar una arquitectura donde todos trabajen de forma coordinada.

---

## Modelo conceptual

---



Cada bloque tiene una responsabilidad específica. El éxito de la solución depende de cómo se combinan, no únicamente de la calidad del modelo.

---

# Tabla de responsabilidades y buenas prácticas por componente

| Componente            | Responsabilidad                                | Buenas prácticas clave                                          |
|-----------------------|------------------------------------------------|-----------------------------------------------------------------|
| Sistema de reglas     | Comportamiento permanente del asistente        | Pocas reglas, claras y estables; separar de capas dinámicas     |
| Contexto de ejecución | Estado actual del usuario y el entorno         | Construir justo antes de la inferencia; validar autenticación   |
| Historial             | Continuidad de la conversación actual          | Definir política de retención; resumir conversaciones extensas  |
| Memoria               | Conocimiento persistente entre conversaciones  | Versionar estructuras; permitir actualización y eliminación     |
| RAG                   | Conocimiento externo recuperado bajo demanda   | Recuperar solo documentos relevantes para la consulta actual    |
| Herramientas          | Información y acciones sobre sistemas externos | Normalizar resultados; manejar errores explícitamente           |
| Políticas y seguridad | Control de acceso y protección de datos        | Validar permisos antes de recuperar; respetar mínimo privilegio |

Prácticas transversales a todos los componentes:

- Registrar el origen de los datos incorporados al contexto.
- Auditar periódicamente la calidad de la información.
- Evitar duplicar información entre componentes.
- Priorizar calidad antes que cantidad.

## Checklist del arquitecto

---

Antes de poner una solución en producción, verifique:

- ¿Las instrucciones del sistema son claras y estables?
- ¿El contexto de ejecución contiene solo información relevante?
- ¿Historial y memoria están correctamente separados?
- ¿RAG recupera únicamente documentos pertinentes?
- ¿Las herramientas validan errores y permisos?
- ¿Se aplican políticas de seguridad antes de construir el contexto?
- ¿Existe una estrategia para controlar el consumo de tokens?

Responder afirmativamente a estas preguntas reduce gran parte de los problemas observados en producción.

---

## Errores de diseño

---

Las implementaciones iniciales suelen presentar alguno de estos problemas:

- utilizar el historial como memoria permanente;
- almacenar documentación completa en memoria;
- mezclar reglas del sistema con datos dinámicos;
- enviar información duplicada al modelo;
- ignorar la seguridad durante la construcción del contexto.

Todos ellos incrementan el costo, reducen la precisión y dificultan el mantenimiento.

---

## Autoevaluación

---

Intente responder las siguientes preguntas sin consultar el material:

1. ¿Qué diferencia existe entre contexto de ejecución y memoria?
2. ¿Qué información debería recuperar RAG?
3. ¿Por qué las herramientas forman parte del contexto?
4. ¿Qué responsabilidad tienen las instrucciones del sistema?
5. ¿Cómo contribuyen las políticas de seguridad a la calidad del contexto?

Si puede responderlas con claridad, dispone de una base sólida para abordar los capítulos siguientes.

---

## Lo que viene

---

En el próximo capítulo estudiaremos las **ventanas de contexto y la gestión de tokens**.

Analizaremos:

- cómo los modelos procesan grandes volúmenes de información;
- límites de las ventanas de contexto;
- estrategias de compresión;
- resumido inteligente;
- optimización del costo y del rendimiento.

Estos conocimientos permitirán diseñar aplicaciones escalables capaces de trabajar con conversaciones y documentos de gran tamaño.

---

## Conclusión

---

La anatomía del contexto constituye el fundamento sobre el que se apoyan todas las aplicaciones modernas de IA. Comprender cada uno de sus componentes y sus responsabilidades permite construir asistentes, agentes y sistemas empresariales más precisos, seguros, mantenibles y escalables.

Con esta base conceptual, ya estamos preparados para estudiar cómo administrar uno de los recursos más valiosos de cualquier modelo de lenguaje: su ventana de contexto.